KCF Object Tracking (No-PTZ)

1. Experiment Objective

The KCF (Kernelized Correlation Filter) tracking algorithm is used to track any object manually selected with a bounding box, and the car rotates to track so that the target stays at the center of the frame.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
KCF Tracking Algorithm	Kernelized Correlation Filter, which uses circulant matrices and the kernel trick to quickly learn the target appearance model
Circulant Matrix Sampling	KCF uses cyclic shifts to generate many training samples and accelerates matrix operations via FFT to achieve high-speed tracking
HOG Features	Histogram of Oriented Gradients, the target appearance feature used by KCF by default
ROI Selection	Drag with the mouse on the image to define the region of interest, used as the initialization target for the tracker
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
  → kcf_tracker_node
    ├── Initialization: select an ROI with the mouse → cv.TrackerKCF_create().init()
    ├── Tracking loop: tracker.update() → new tracking box → centroid calculation
    └── Body control timer (10Hz): pixel error → PD angular velocity → /cmd_vel (BEST_EFFORT)
```

> **Note:** This feature only supports track (tracking) mode; follow and control modes are not supported. Once the KCF tracker loses the target, it cannot recover automatically.

3. Core Topic Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
`/cmd_vel`	`geometry_msgs/Twist`	best_effort + keep_last(1)	`angular.z`	-0.60~0.60 rad/s	`+=` turn left, `-=` turn right

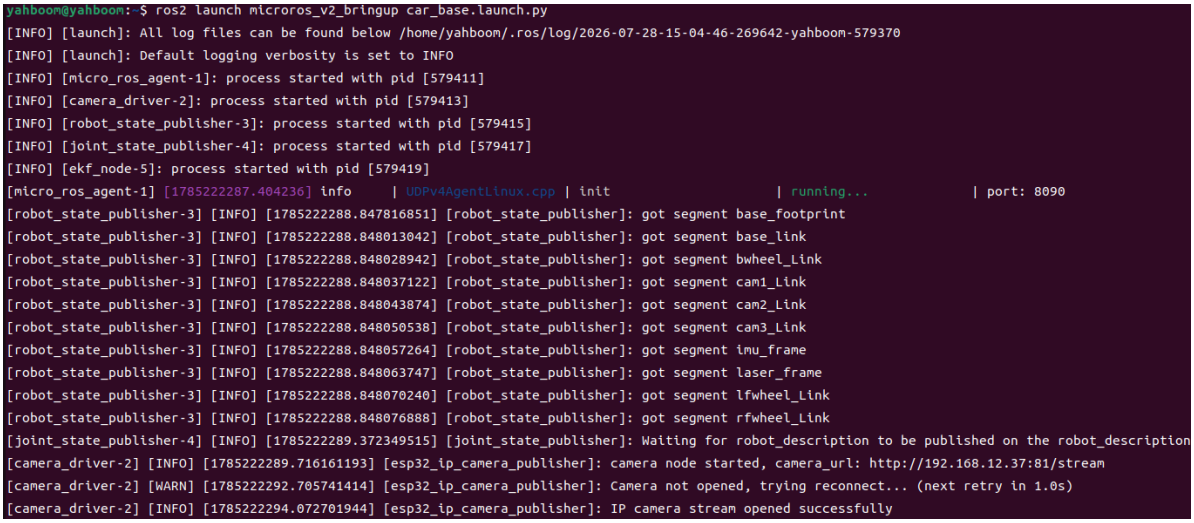
3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	❌
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted)

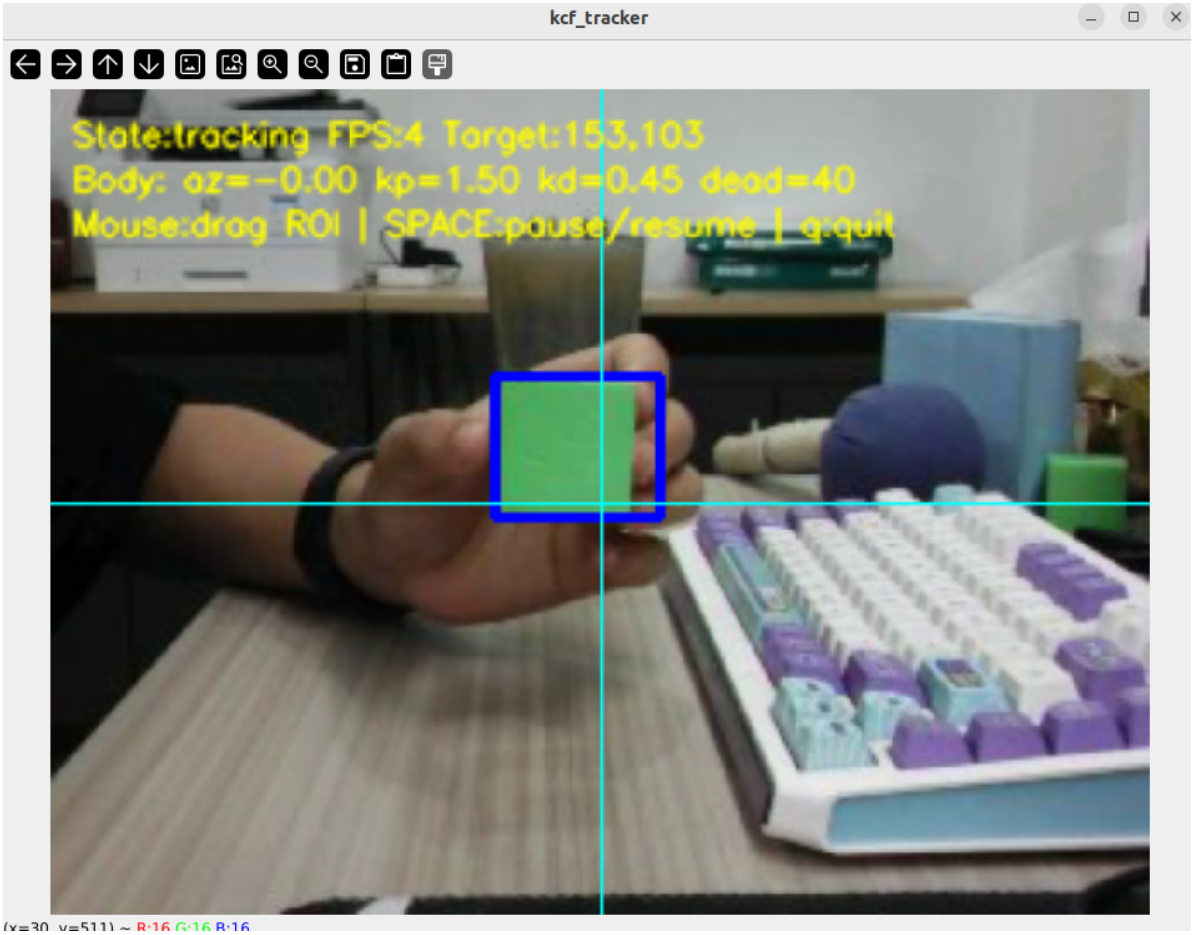
2. Start the Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```



3. Start KCF Object Tracking (Terminal 2)

```
ros2 launch microros_v2_no_ptz_visual_control_pkg kcf_tracker.launch.py
```



5. Operation Instructions

Key	Function
Space	Confirm the selection and start tracking / toggle the tracking state
q / Q / ESC	Exit the program

Operation	Function
Drag with the left mouse button	Drag to select the target object, then press Space to confirm and start tracking

4. Experiment Phenomenon

- After selecting a target, the tracking box follows the target and the car automatically rotates to keep the target at the center of the frame
- After the target moves out of the frame or is fully occluded, the frame shows "Target Lost!"
- The KCF tracker cannot recover automatically after the target reappears; you need to manually re-select the target
- The ESP32 camera has a low frame rate (about 7 fps); it is recommended to select a larger, feature-rich target and move it slowly

5. Code Explanation

> Source path:
>
> - Launch: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/kcf_tracker`

> `.launch.py`
> - Node source: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/kcf_tracker_node.py`

1. Core Node Analysis

`kcf_tracker_node` uses the OpenCV Tracker API. Core workflow:

- **Initialization**: waits for the user to select an ROI with the mouse → press Space → create `cv.TrackerKCF_create()` (or MIL/CSRT) → `tracker.init(frame, bbox)`
- **Tracking loop** (15Hz): each frame `ok, bbox = tracker.update(frame)` → on success, update the tracking box position + compute the centroid deviation → PD outputs angular velocity
- **Loss detection**: `ok=False` or `bbox` out of the frame → mark as lost (Lost), wait for re-selection
- **KCF limitation**: no automatic re-detection after loss; the user must manually re-initialize

2. Key Parameters

> Parameter definition file:
>
> - Launch: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/kcf_tracker.launch.py`
> - Node: `~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/kcf_tracker_node.py`

Parameter	Type	Default	Description
`tracker_type`	str	`KCF`	Tracking algorithm type (KCF / MIL / CSRT)
`deadband_x_pixel`	float	`40.0` px	Horizontal deadband
`angular_kp`	float	`1.5`	Body angle PD proportional coefficient
`angular_kd`	float	`0.45`	Body angle PD derivative coefficient
`max_angular_z`	float	`0.6` rad/s	Maximum body steering speed
`reverse_body_direction`	bool	`true`	Whether to reverse the body steering direction
`image_processing_rate_hz`	float	`15.0` Hz	Image processing rate
`body_control_rate`	float	`10.0` Hz	Body control rate

3. Node Description

Node	Package	Description
`kcf_tracker_node`	`microros_v2_no_ptz_visual_control_pkg`	KCF object tracking with mouse selection + PD body control

6. Common Problems

Problem	Cause	Solution
Unstable tracking	Low frame rate or the target is too small	Select a large target (10%–30% of the frame) and slow down the target movement
Cannot recover after tracking loss	KCF does not re-detect automatically	Manually re-select the target after loss
Tracking box drift	The target appearance changes beyond the model's adaptation range	Select an object with distinct texture features